

APPLICATION DEVELOPMENT AND SECURE CODING GUIDELINES

Ganesh Abdagire

Technical Architect

GEBBS healthcare solutions

Ganesh.Abdagire@gebbs.com

Why do we need secure coding guidelines?

Input Validation

Output Encoding

Authentication and Password Management

Session management

Cryptographic practices

Error handling and logging

Data protection

Communication Security

DISCUSSION PLAN

System Configuration

Database Security

File Management

Memory Management

General Coding Practices

Clickjacking

CSRF

AI Security practices

DISCUSSION PLAN

WHY DO WE NEED SECURE CODING GUIDELINES?

Removes
commonly
exploited software
vulnerabilities.

Prevents
cyberattacks from
happening.



DATA PROTECTION



Encrypt sensitive data both at rest (stored data) and in transit (data being transmitted over networks).



Protect server-side source code from being downloaded by a user.



Do not store passwords, connection strings or other sensitive information in clear text or in any non-cryptographically secure manner on the client side.



Remove comments in user-accessible production code that may reveal backend system or other sensitive information.



Do not include sensitive information in HTTP GET request parameters.

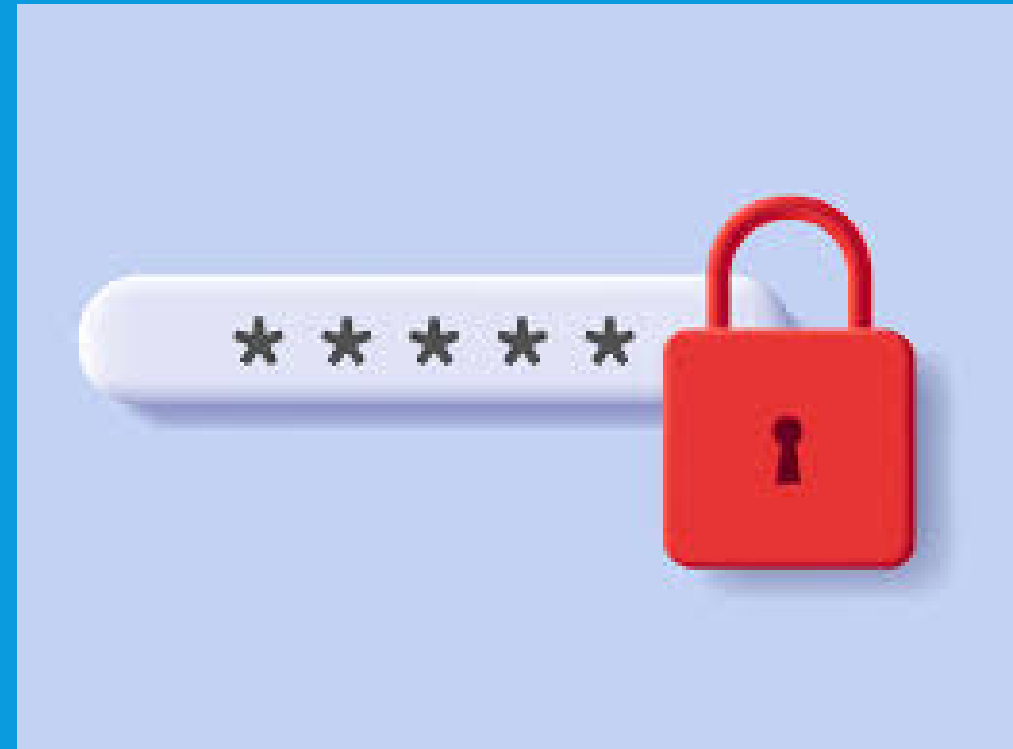


Disable client-side caching on pages containing sensitive information.
Cache-Control: no-store, may be used in conjunction with the HTTP header control "Pragma: no-cache", which is less effective, but is HTTP/1.0 backward compatible



AUTHENTICATION AND PASSWORD MANAGEMENT

- Password hashing must be implemented on a trusted system (e.g., The server).
- Enforce password complexity requirements established by policy or regulation.
- Authentication failure responses should not indicate which part of the authentication data was incorrect. For example, instead of "Invalid username" or "Invalid password", just use "Invalid username and/or password" for both. Error responses must be truly identical in both display and source code.
- Use only HTTP POST requests to transmit authentication credentials.
- Disable the "remember me" functionality for password fields.
- Disable autocompletion for login forms.



ERROR HANDLING AND LOGGING

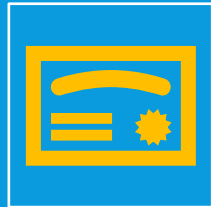
Do not disclose	Do not disclose sensitive information in error responses, including system details, session identifiers or account information.
Use	Use error handlers that do not display debugging or stack trace information.
Implement	Implement generic error messages and use custom error pages.
Restrict	Restrict access to logs to only authorized individuals.
Log	Log all input validation failures.
Log	Log all apparent tampering events, including unexpected changes to state data



COMMUNICATION SECURITY



Implement encryption for the transmission of all sensitive information. For example use HTTPS protocol for transmission.



TLS certificates should be valid and have the correct domain name, not be expired, and be installed with intermediate certificates when required.

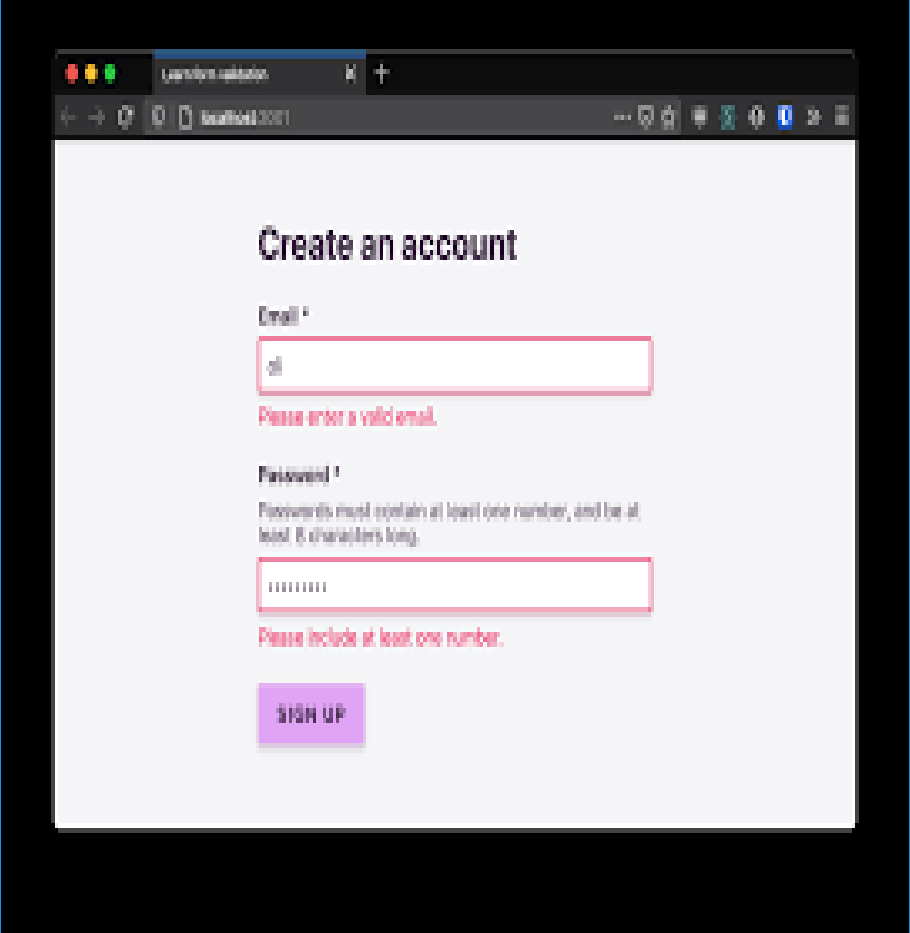


Failed TLS connections should not fall back to an insecure connection



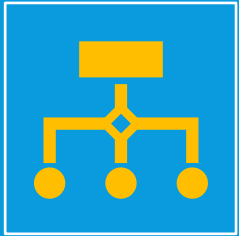
INPUT VALIDATION

Conduct	Conduct all data validation on a trusted system (e.g., The server)
Specify	Specify proper character sets, such as UTF-8, for all sources of input.
Encode	Encode data to a common character set before val.
Validate	Validate for expected data types and data length.
Be	If any potentially hazardous characters must be allowed as input, be sure that you implement additional controls like output encoding. Example of characters <code>< > ' " % () & + \ \\'</code>



The screenshot shows a web browser window with the title 'Create an account'. The form contains two input fields: 'Email' and 'Password'. The 'Email' field has a red border and a red error message below it: 'Please enter a valid email.' The 'Password' field also has a red border and a red error message below it: 'Passwords must contain at least one number, and be at least 8 characters long.' At the bottom of the form is a purple button labeled 'SIGN UP'.

ACCESS CONTROL



Principle of Least Privilege: Ensure users and processes only have access to the data they need to perform their specific job functions.



Role-Based Access Control (RBAC): Implement RBAC to limit access based on user roles.

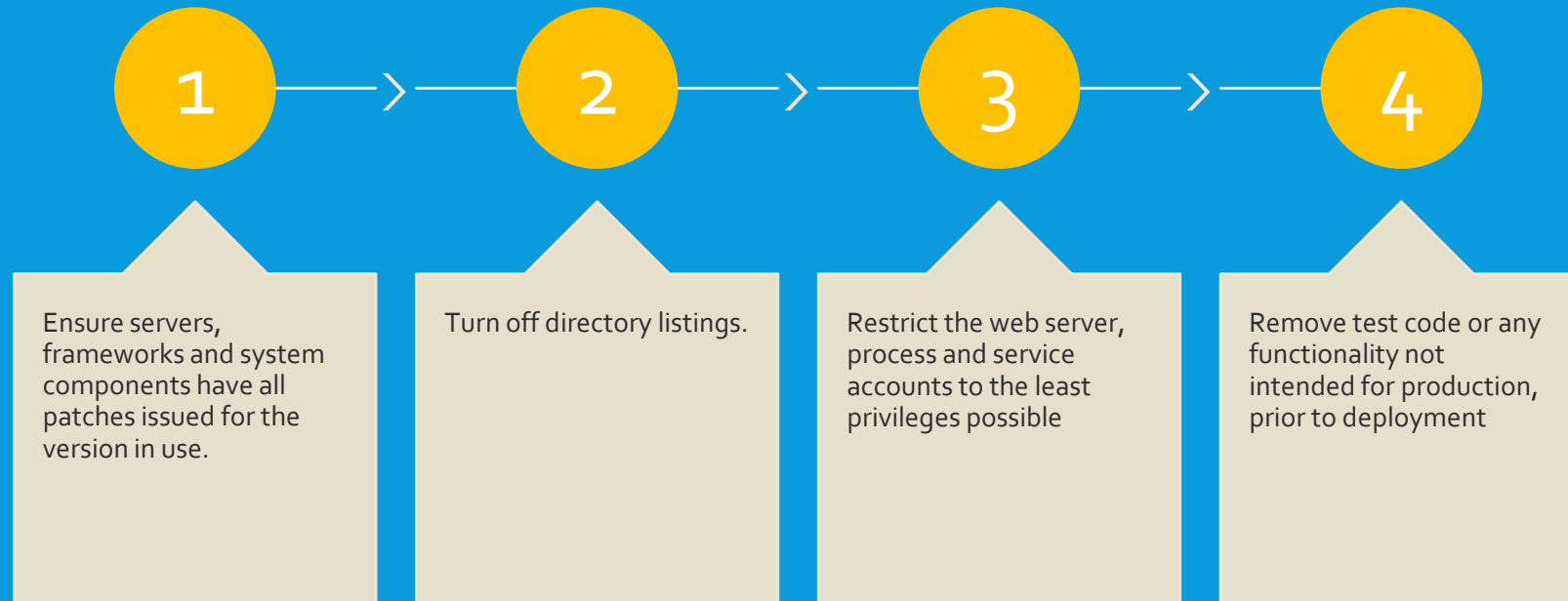


Strong Authentication: Use multi-factor authentication (MFA) for users accessing sensitive health information.



Session Management: Implement secure session management (e.g., automatic session timeouts, secure cookies, and token-based authentication).

SYSTEM CONFIGURATION



DATABASE SECURITY

Use secure credentials for database access.

Connection strings should not be hard coded within the application.

Connection strings should be stored in a separate configuration file on a trusted system and they should be encrypted.

Use stored procedures to abstract data access and allow for the removal of permissions to the base tables in the database.

Close the connection as soon as possible.

Disable any default accounts that are not required to support business requirements

Database
Security
Best
Practices



SQL INJECTION

DO: Using an object relational mapper (ORM) or stored procedures is the most effective way of countering the SQL Injection vulnerability.

DO: Use parameterized queries where a direct SQL query must be used.

DO NOT: Concatenate strings anywhere in your code and execute them against your database (Known as *dynamic SQL*).

```
string sql = "SELECT * FROM Users WHERE UserName='" + txtUser.Text + "' AND Password='"
```

```
+ txtPassword.Text + "'";  
context.Database.ExecuteSqlCommand(sql); // SQL Injection vulnerability!
```

SOFTWARE AND DATA INTEGRITY FAILURES



DO: Digitally sign assemblies and executable files.



DO: Use Nuget package signing.



DO NOT: Send unsigned or unencrypted serialized objects over the network



DO NOT: Use the BinaryFormatter type which is dangerous and not recommended for data processing.

Clickjacking is a type of cyber attack where a malicious website or element is used to trick a user into clicking something different from what the user perceives, effectively hijacking their clicks.

How Clickjacking Works:

Malicious Site Setup: The attacker creates a web page that contains hidden or transparent frames (iframes) over legitimate content from another site, such as a social media button or a financial transaction button.

Deceptive Appearance: The attacker positions these hidden elements so that the page looks normal to the user, such as a button they might want to click.

User Interaction: When the user clicks on what they think is an innocuous button (like "Play Video" or "Like" on a social media platform), they are actually clicking on the hidden frame, which performs a malicious action.

CLICKJACKING

CROSS-SITE REQUEST FORGERY

- **Anti-forgery** refers to the protection mechanisms put in place to prevent **forgery attacks**, particularly **Cross-Site Request Forgery (CSRF)**, where an attacker tricks a user into performing actions they did not intend to on a trusted website, using their authentication credentials.
- **How to Mitigate CSRF Attacks (Anti-Forgery Mechanisms)**
 - **Anti-CSRF Tokens** (also known as **CSRF Tokens** or **Forgery Tokens**)
 - **SameSite Cookies: (SameSite=Strict)** Cookies are only sent if the request originates from the same site.
 - **Requiring Authentication for Critical Actions:**
 - Use CAPTCHAs

AWS- IDENTITY & CREDENTIALS

Never hardcode secrets or credentials. Use AWS Secrets Manager / SSM Parameter Store or IRSA for EKS.

Use IAM Roles for Service Accounts (IRSA) in EKS — bind Kubernetes service accounts to fine-grained IAM roles instead of embedding keys.

Separate task/execution roles in ECS. TaskRole for app permissions; ExecutionRole for image pulls/logging.

Principle of least privilege: craft narrow IAM policies; audit and remove unused permissions regularly.

Example IRSA annotation (K8s service account):

AWS- SECRETS & CONFIGURATION

Store	Store secrets centrally (Secrets Manager / Parameter Store). Do not commit .env or secrets to source control.
Inject	Inject secrets at runtime (K8s Secrets mounted as volumes or env via CSI driver) — prefer volume-mounted secrets for larger payloads.
Enable	Enable automatic secret rotation where possible and ensure apps can handle secret rotation (retry/refresh).
Fetch	Example: fetch secret via AWS SDK (pseudo):

AWS- DOCKERFILE & IMAGE BEST PRACTICES

Use minimal base images (distroless, alpine, or specific slim images). Avoid ubuntu:latest.

Pin base image versions (no latest). e.g., FROM python:3.11.4-slim.

Multi-stage builds to keep runtime images small and remove build-time artifacts.

Run as non-root user inside container.

Remove build-time secrets and .git metadata before final image.

Scan images on build (Trivy/Grype/ECR scanning) and fail builds on high/CVSS > threshold.

Example Dockerfile pattern:

AWS- LOGGING, TELEMETRY & PII HANDLING

Do not log PHI. Mask or redact sensitive fields before writing logs.

Structured logs (JSON) with standardized fields: service, env, trace_id, user_id_masked.

Avoid logging entire request bodies — log metadata and hashes only.

Implement log-level gating (lower levels for prod; dev has more verbose).

Example redaction pseudocode:

AWS- CODE LEVEL CONTROLS (INPUT/OUTPUT)

Sanitize	Sanitize all external input (validate types, lengths, content) before using in prompts or DB calls.
Implement	PII/PHI detection & redaction: implement a lightweight PHI classifier to block or redact PHI before sending to any model or external API.
Send	Never send raw PHI to third-party LLMs unless covered by BAA and approved architecture. If allowed, always use private endpoints and encryption.
Escape	Escape outputs before rendering in UIs (prevent XSS) and remove sensitive tokens before logging.

Layer	Control	Example
Data	De-identification	Replace patient name with pseudonym
Model	Guardrails	Pre-prompt filter for PHI terms
API	Logging control	Mask all PHI before storing
Storage	Encryption	S3 buckets with SSE-KMS
Access	IAM	Least privilege for model users
Audit	Logging	Immutable CloudTrail + SIEM integration

HIPAA CONTROLS FOR AN AI AGENT



Never use production PHI in local dev/test environments.



Implement **PHI classifiers** to detect sensitive fields in prompts.



Maintain **secure context isolation** for AI sessions.



Use **human-in-loop review** for AI outputs that impact patients.



Periodically run **penetration tests** data leaks.

DEVELOPER BEST PRACTICES

THANK YOU